

Experiment-12

Name: - C. Jyothi

Regno: -192425312

Course Code: -MLA0305

Slot: B

Title: Implementation of the Proximal Policy Optimization Algorithm

Aim:

To implement PPO and evaluate its stability and performance in discrete and continuous action environments.

Procedure:

1. Select a suitable discrete or continuous environment.
2. Create Actor and Critic neural networks.
3. Collect trajectories using the current policy.
4. Calculate returns and advantages.
5. Calculate the probability ratio between new and old policies.
6. Apply PPO's clipped objective function.
7. Update the Actor and Critic networks.
8. Repeat the training process for several iterations.
9. Record cumulative rewards and losses.
10. Evaluate policy stability and performance.



The screenshot shows a Jupyter Notebook interface. The top bar includes a menu (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for commands, code, text, and running. Below the toolbar, there's a search bar and a 'Run all' button. The main area contains a code cell with the following Python code:

```
[4]: import numpy as np
old=np.array([.4,.3,.5,.25]); new=np.array([.44,.24,.7,.2]); adv=np.array([1,1,-1,-1]); eps=.2
ratio=new/old; clip=np.clip(ratio,1-eps,1+eps); obj=np.minimum(ratio*adv,clip*adv)
print('Ratio:',ratio); print('Clipped:',clip); print('PPO objective:',obj); print('Mean:',obj.mean())
```

The output of the code cell is displayed below the code:

```
... Ratio: [1.1 0.8 1.4 0.8]
Clipped: [1.1 0.8 1.2 0.8]
PPO objective: [ 1.1  0.8 -1.4 -0.8]
Mean: -0.07500000000000001
```

Result:

The PPO algorithm successfully learned an effective policy. The clipping mechanism provided stable policy updates and improved training stability.